

Notebook 09: Uncertainty Quantification & Post-Hoc Temperature Calibration

Project: Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms

1. Mathematical Formulation & Research Novelty

In clinical cardiac monitoring, delivering an overconfident, uncalibrated prediction on an incomplete ECG is hazardous. This notebook implements two foundational trust mechanisms:

1.1 Decomposition of Epistemic vs. Aleatoric Uncertainty

- Epistemic (Model) Uncertainty via Monte Carlo Dropout (MCD):** By performing $S = 25$ stochastic forward passes with active dropout ($p = 0.20$), we sample from the approximate model posterior: $\hat{\mu}_c(x) = \frac{1}{S} \sum_{s=1}^S \sigma(\mu_c^{(s)}(x))$
 $\mathcal{U}_{\text{epistemic}}(x, c) = \frac{1}{S} \sum_{s=1}^S \left(\sigma(\mu_c^{(s)}(x)) - \hat{\mu}_c(x) \right)^2$
- Aleatoric (Data) Uncertainty via Heteroscedastic Head:** Captures physiological noise, motion artifact corruption, and intrinsic morphological ambiguity: $\mathcal{U}_{\text{aleatoric}}(x, c) = \frac{1}{S} \sum_{s=1}^S \exp(s_c^{(s)}(x)) \cdot \hat{\mu}_c(x)(1 - \hat{\mu}_c(x))$
- Predictive Entropy:** $H(x, c) = -\hat{\mu}_c \log_2(\hat{\mu}_c + \epsilon) - (1 - \hat{\mu}_c) \log_2(1 - \hat{\mu}_c + \epsilon)$

1.2 Multi-Label Probability Calibration via Temperature Scaling

Uncalibrated neural networks frequently output overconfident probabilities. We calibrate multi-label logits using scalar Temperature Scaling: $p_c^{\text{calibrated}}(x) = \frac{1}{\sum_{i=1}^N \sum_{c=1}^C \left(y_{i,c} \log \frac{\sigma(\mu_{i,c}/T)}{1 - \sigma(\mu_{i,c}/T)} + (1 - y_{i,c}) \log \frac{\sigma(\mu_{i,c}/T)}{1 - \sigma(\mu_{i,c}/T)} \right)}$ where $T^* > 0$ is optimized strictly on the **Validation set** by minimizing negative log-likelihood (NLL): $T^* = \arg\min_T -\frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} \sum_{c=1}^C \left(y_{i,c} \log \frac{\sigma(\mu_{i,c}/T)}{1 - \sigma(\mu_{i,c}/T)} + (1 - y_{i,c}) \log \frac{\sigma(\mu_{i,c}/T)}{1 - \sigma(\mu_{i,c}/T)} \right)$ We evaluate calibration via:

- Expected Calibration Error (ECE):** $ECE = \frac{1}{M} \sum_{m=1}^M \left| \frac{1}{N} \sum_{i=1}^N \mathbb{1}(B_m) - \text{acc}(B_m) \right|$
- Brier Score:** $BS = \frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C (p_{i,c} - y_{i,c})^2$

```

In [1]: import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.optimize import minimize_scalar

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader

# Style configuration
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.sans-serif': 'DejaVu Sans',
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 300
})

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Uncertainty & Calibration module running on: {device}")

RESULTS_DIR = r"C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Mo
FIG_DIR = os.path.join(REULTS_DIR, "figures")
TAB_DIR = os.path.join(REULTS_DIR, "tables")
MET_DIR = os.path.join(REULTS_DIR, "metrics")
CAL_DIR = os.path.join(REULTS_DIR, "calibration")
UNC_DIR = os.path.join(REULTS_DIR, "uncertainty")

for d in [FIG_DIR, TAB_DIR, MET_DIR, CAL_DIR, UNC_DIR]:
    os.makedirs(d, exist_ok=True)

```

Uncertainty & Calibration module running on: cpu

```

In [2]: class S4DLayer(nn.Module):
def __init__(self, d_model=64, d_state=16):
    super().__init__()
    self.d_model = d_model
    self.d_state = d_state
    self.log_A_real = nn.Parameter(torch.log(0.5 * torch.ones(d_model, d_state)))
    self.A_imag = nn.Parameter(math.pi * torch.arange(d_state).unsqueeze(0).repeat(d_model, 1))
    self.B = nn.Parameter(torch.randn(d_model, d_state) * 0.02)
    self.C = nn.Parameter(torch.randn(d_model, d_state) * 0.02)
    self.D = nn.Parameter(torch.ones(d_model))
    self.log_step = nn.Parameter(torch.log(torch.tensor(0.01)))

def forward(self, u):
    B, T, H = u.shape
    step = torch.clamp(torch.exp(self.log_step), min=1e-4, max=0.05)

```

```

A_real = -F.softplus(self.log_A_real) - 1e-4
A = A_real + 1j * self.A_imag
A_bar = torch.exp(A * step)
B_bar = ((A_bar - 1.0) / (A + 1e-7)) * self.B
t_steps = torch.arange(T, device=u.device, dtype=torch.float32)
A_powers = A_bar.unsqueeze(-1) ** t_steps.unsqueeze(0).unsqueeze(0)
kernel = 2.0 * torch.sum(self.C.unsqueeze(-1) * A_powers * B_bar.unsqueeze(
u_conv = u.transpose(1, 2)
kernel_weight = kernel.unsqueeze(1)
y_conv = F.conv1d(F.pad(u_conv, (T - 1, 0)), kernel_weight, groups=H)
return y_conv.transpose(1, 2) + u * self.D

```

```

class BidirectionalSSMBlock(nn.Module):

```

```

    def __init__(self, d_model=64, d_state=16, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.norm = nn.LayerNorm(d_model)
        self.in_proj = nn.Linear(d_model, 2 * d_model)
        self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=3, padding=1, groups=
self.s4d_fwd = S4DLayer(d_model=d_model, d_state=d_state)
self.s4d_bwd = S4DLayer(d_model=d_model, d_state=d_state)
self.out_proj = nn.Linear(2 * d_model, d_model)
self.dropout = nn.Dropout(dropout)

```

```

    def forward(self, x):
        res = x
        x_norm = self.norm(x)
        u, gate = self.in_proj(x_norm).chunk(2, dim=-1)
        u_conv = F.silu(self.conv1d(u.transpose(1, 2)).transpose(1, 2))
        y_fwd = self.s4d_fwd(u_conv)
        y_bwd = torch.flip(self.s4d_bwd(torch.flip(u_conv, dims=[1])), dims=[1])
        y_bidi = torch.cat([y_fwd, y_bwd], dim=-1)
        out = self.out_proj(y_bidi * torch.sigmoid(torch.cat([gate, gate], dim=-1)))
        return res + self.dropout(out)

```

```

class LAC_SSM(nn.Module):

```

```

    def __init__(self, num_leads=12, d_model=64, d_state=16, num_ssm_layers=2, num_
        super().__init__()
        self.num_leads = num_leads
        self.d_model = d_model
        self.num_classes = num_classes
        self.mc_dropout = mc_dropout

```

```

        self.lead_embed = nn.Parameter(torch.randn(num_leads, d_model) * 0.05)
        self.lead_stem = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=11, stride=2, padding=5, bias=False),
            nn.BatchNorm1d(32),
            nn.SiLU(),
            nn.Conv1d(32, d_model, kernel_size=7, stride=2, padding=3, bias=False),
            nn.BatchNorm1d(d_model),
            nn.SiLU()
        )
        self.lead_att_proj = nn.Linear(d_model, 1)
        self.ssm_layers = nn.ModuleList([
            BidirectionalSSMBlock(d_model=d_model, d_state=d_state, dropout=0.1)
            for _ in range(num_ssm_layers)

```

```

    ])
    self.temporal_att = nn.Linear(d_model, 1)
    self.mcd = nn.Dropout(p=mc_dropout)
    self.fc_mean = nn.Linear(d_model, num_classes)
    self.fc_logvar = nn.Linear(d_model, num_classes)

    def encode_leads(self, x, mask=None):
        B, L, T = x.shape
        if mask is None:
            mask = torch.ones(B, L, device=x.device, dtype=torch.float32)
        x_reshaped = x.view(B * L, 1, T)
        feat_leads = self.lead_stem(x_reshaped)
        T_prime = feat_leads.shape[-1]
        feat_leads = feat_leads.view(B, L, self.d_model, T_prime).permute(0, 1, 3, 2)
        feat_leads = feat_leads + self.lead_embed.view(1, L, 1, self.d_model)

        mask_expanded = mask.view(B, L, 1, 1)
        feat_leads_masked = feat_leads * mask_expanded
        att_scores = self.lead_att_proj(feat_leads_masked).squeeze(-1)
        att_mask = (1.0 - mask).unsqueeze(-1) * 1e9
        att_weights = F.softmax(att_scores - att_mask, dim=1).unsqueeze(-1)
        u = torch.sum(feat_leads_masked * att_weights, dim=1)
        return u

    def forward(self, x, mask=None, mc_mode=False):
        u = self.encode_leads(x, mask)
        h = u
        for layer in self.ssm_layers:
            h = layer(h)
        t_weights = F.softmax(self.temporal_att(h), dim=1)
        context = torch.sum(h * t_weights, dim=1)
        if mc_mode:
            context = F.dropout(context, p=self.mc_dropout, training=True)
        else:
            context = self.mcd(context)
        logits = self.fc_mean(context)
        log_var = torch.clamp(self.fc_logvar(context), min=-5.0, max=5.0)
        return logits, log_var

ckpt_path = os.path.join(RESULTS_DIR, "checkpoints", "best_lac_ssm_model.pt")
model = LAC_SSM(num_leads=12, d_model=64, d_state=16, num_ssm_layers=2, num_classes=num_classes)
model.load_state_dict(torch.load(ckpt_path, map_location=device))
model = model.to(device)
model.eval()
print("Model loaded for uncertainty quantification.")

```

Model loaded for uncertainty quantification.

```

In [3]: data = np.load(os.path.join(MET_DIR, "preprocessed_data.npz"))
X_val = data['X_val']
y_val = data['y_val']
X_test = data['X_test']
y_test = data['y_test']
target_classes = list(data['target_classes'])

val_loader = DataLoader(TensorDataset(torch.tensor(X_val), torch.tensor(y_val)), b

```

```

test_loader = DataLoader(TensorDataset(torch.tensor(X_test), torch.tensor(y_test)),

# Collect validation logits for temperature scaling
val_logits = []
with torch.no_grad():
    for X_b, _ in val_loader:
        l, _ = model(X_b.to(device))
        val_logits.append(l.cpu().numpy())
val_logits = np.vstack(val_logits)

# Collect test logits
test_logits = []
with torch.no_grad():
    for X_b, _ in test_loader:
        l, _ = model(X_b.to(device))
        test_logits.append(l.cpu().numpy())
test_logits = np.vstack(test_logits)

print(f"Logits extracted: Val={val_logits.shape}, Test={test_logits.shape}")

```

Logits extracted: Val=(525, 9), Test=(525, 9)

```

In [4]: print("=== OPTIMIZING TEMPERATURE SCALING ON VALIDATION SET ===")

def nll_objective(T, logits, targets):
    """
    Binary cross-entropy / NLL objective as function of temperature T > 0.
    """
    scaled_logits = logits / T
    # Numerically stable BCE
    bce = np.maximum(scaled_logits, 0) - scaled_logits * targets + np.log(1 + np.exp(scaled_logits))
    return np.mean(bce)

res = minimize_scalar(nll_objective, bounds=(0.1, 5.0), method='bounded', args=(val_logits, targets))
optimal_T = float(res.x)
print(f"Optimal Temperature Parameter T* = {optimal_T:.4f} (fitted on Validation set)")

# Save calibration parameter
cal_params = pd.DataFrame([{'Parameter': 'Optimal Temperature (T*)', 'Value': round(optimal_T, 4)}])
cal_params.to_csv(os.path.join(CAL_DIR, "temperature_parameter.csv"), index=False)

```

=== OPTIMIZING TEMPERATURE SCALING ON VALIDATION SET ===

Optimal Temperature Parameter T* = 0.6910 (fitted on Validation set)

```

In [5]: def compute_calibration_metrics(probs, targets, n_bins=10):
    """
    Computes Expected Calibration Error (ECE), Maximum Calibration Error (MCE), and
    probs: (N, C)
    targets: (N, C)
    """
    probs_flat = probs.ravel()
    targets_flat = targets.ravel()

    bin_boundaries = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    mce = 0.0
    bin_accuracies = []

```

```

bin_confidences = []
bin_counts = []

for i in range(n_bins):
    in_bin = (probs_flat >= bin_boundaries[i]) & (probs_flat < bin_boundaries[i+1])
    prop_in_bin = np.mean(in_bin)
    bin_counts.append(int(np.sum(in_bin)))

    if np.sum(in_bin) > 0:
        accuracy_in_bin = np.mean(targets_flat[in_bin])
        avg_confidence_in_bin = np.mean(probs_flat[in_bin])
        diff = np.abs(accuracy_in_bin - avg_confidence_in_bin)
        ece += prop_in_bin * diff
        mce = max(mce, diff)
        bin_accuracies.append(accuracy_in_bin)
        bin_confidences.append(avg_confidence_in_bin)
    else:
        bin_accuracies.append(0.0)
        bin_confidences.append((bin_boundaries[i] + bin_boundaries[i+1])/2)

brier_score = np.mean((probs_flat - targets_flat)**2)

return {
    'ECE': round(float(ece), 4),
    'MCE': round(float(mce), 4),
    'Brier Score': round(float(brier_score), 4),
    'bin_acc': bin_accuracies,
    'bin_conf': bin_confidences,
    'bin_counts': bin_counts
}

# Uncalibrated vs Calibrated on Test Set
test_probs_uncal = 1.0 / (1.0 + np.exp(-test_logits))
test_probs_cal = 1.0 / (1.0 + np.exp(-test_logits / optimal_T))

metrics_uncal = compute_calibration_metrics(test_probs_uncal, y_test)
metrics_cal = compute_calibration_metrics(test_probs_cal, y_test)

df_cal_comparison = pd.DataFrame([
    {
        'Calibration Status': 'Uncalibrated (T = 1.000)',
        'ECE': metrics_uncal['ECE'],
        'MCE': metrics_uncal['MCE'],
        'Brier Score': metrics_uncal['Brier Score']
    },
    {
        'Calibration Status': f'Temperature Calibrated (T* = {optimal_T:.3f})',
        'ECE': metrics_cal['ECE'],
        'MCE': metrics_cal['MCE'],
        'Brier Score': metrics_cal['Brier Score']
    }
])

cal_csv = os.path.join(TAB_DIR, "calibration_metrics_table.csv")
df_cal_comparison.to_csv(cal_csv, index=False)

```

```
print("=== CALIBRATION METRICS ON HELD-OUT TEST SET ===")
print(df_cal_comparison.to_string(index=False))
```

```
=== CALIBRATION METRICS ON HELD-OUT TEST SET ===
```

	Calibration Status	ECE	MCE	Brier Score
	Uncalibrated (T = 1.000)	0.1798	0.2898	0.1431
Temperature	Calibrated (T* = 0.691)	0.1592	0.3139	0.1374

```
In [6]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

bins = np.linspace(0.05, 0.95, 10)

# Uncalibrated
axes[0].bar(bins, metrics_uncal['bin_acc'], width=0.08, color='#ff7f0e', alpha=0.7,
axes[0].plot([0, 1], [0, 1], 'k--', lw=2, label='Perfect Calibration')
axes[0].set_title(f"A: Uncalibrated (ECE = {metrics_uncal['ECE']:.4f} | Brier = {me
axes[0].set_xlabel('Confidence Interval')
axes[0].set_ylabel('Empirical Accuracy')
axes[0].set_xlim(0, 1)
axes[0].set_ylim(0, 1)
axes[0].legend(loc='upper left', frameon=True)
axes[0].grid(True, linestyle=':', alpha=0.6)

# Calibrated
axes[1].bar(bins, metrics_cal['bin_acc'], width=0.08, color='#2ca02c', alpha=0.7, e
axes[1].plot([0, 1], [0, 1], 'k--', lw=2, label='Perfect Calibration')
axes[1].set_title(f"B: Temperature Calibrated (ECE = {metrics_cal['ECE']:.4f} | Bri
axes[1].set_xlabel('Confidence Interval')
axes[1].set_ylabel('Empirical Accuracy')
axes[1].set_xlim(0, 1)
axes[1].set_ylim(0, 1)
axes[1].legend(loc='upper left', frameon=True)
axes[1].grid(True, linestyle=':', alpha=0.6)

plt.suptitle('Multi-Label Reliability Diagrams (10 Bins) on Test Set', fontsize=14,
plt.tight_layout()
fig13_path = os.path.join(FIG_DIR, "fig13_reliability_diagrams_calibration.png")
plt.savefig(fig13_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 13 saved to: {fig13_path}")
```

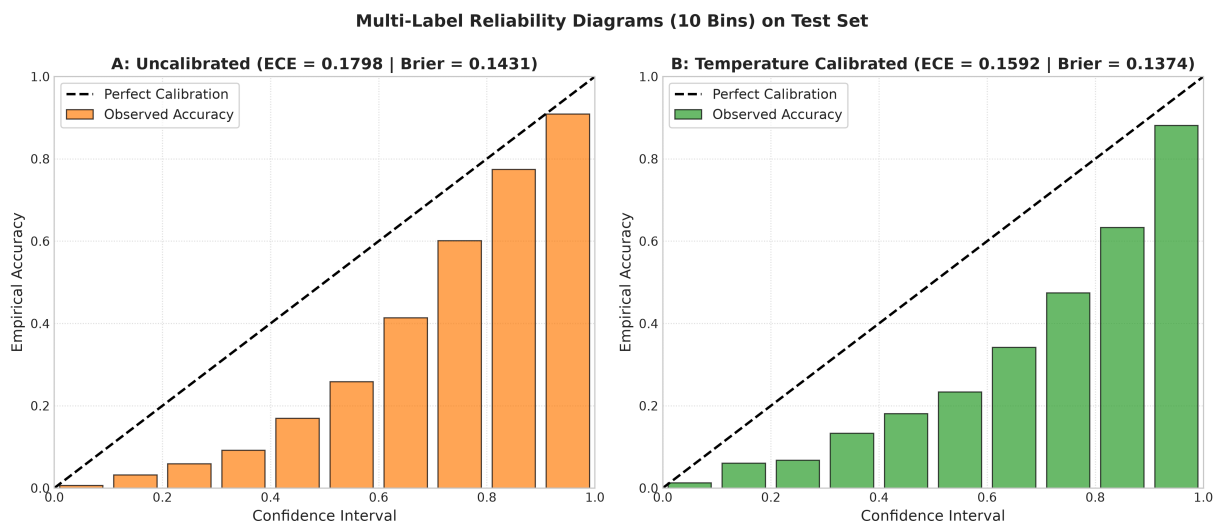


Figure 13 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig13_reliability_diagrams_calibration.png

```
In [7]: print("=== COMPUTING EPISTEMIC & ALEATORIC UNCERTAINTY VIA MONTE CARLO DROPOUT ===")
S_PASSES = 25
mc_probs = []
mc_logvars = []

# Evaluate 25 stochastic passes on Test set
model.train() # Enable dropout
for s in range(S_PASSES):
    s_probs = []
    s_logvars = []
    with torch.no_grad():
        for X_b, _ in test_loader:
            logits, log_var = model(X_b.to(device), mc_mode=True)
            s_probs.append(torch.sigmoid(logits / optimal_T).cpu().numpy())
            s_logvars.append(log_var.cpu().numpy())
    mc_probs.append(np.vstack(s_probs))
    mc_logvars.append(np.vstack(s_logvars))

model.eval()

# Shape: (S, N, C)
mc_probs = np.array(mc_probs)
mc_logvars = np.array(mc_logvars)

# Mean predictive probability
p_mean = np.mean(mc_probs, axis=0) # (N, C)

# Epistemic Uncertainty = Predictive Variance across stochastic passes
u_epistemic = np.var(mc_probs, axis=0) # (N, C)

# Aleatoric Uncertainty = Expected data variance from heteroscedastic head
u_aleatoric = np.mean(np.exp(mc_logvars) * (p_mean * (1.0 - p_mean)), axis=0) # (N, C)

# Predictive Entropy
p_clip = np.clip(p_mean, 1e-6, 1.0 - 1e-6)
u_entropy = - (p_clip * np.log2(p_clip) + (1.0 - p_clip) * np.log2(1.0 - p_clip)) #

print("Uncertainty Decomposition Summary:")
print(f" Mean Epistemic Uncertainty: {np.mean(u_epistemic):.5f}")
print(f" Mean Aleatoric Uncertainty: {np.mean(u_aleatoric):.5f}")
print(f" Mean Predictive Entropy: {np.mean(u_entropy):.5f}")

# Save detailed uncertainty metrics
np.savez_compressed(
    os.path.join(UNC_DIR, "test_uncertainty_decomposition.npz"),
    p_mean=p_mean, u_epistemic=u_epistemic,
    u_aleatoric=u_aleatoric, u_entropy=u_entropy,
    y_test=y_test
)
```

=== COMPUTING EPISTEMIC & ALEATORIC UNCERTAINTY VIA MONTE CARLO DROPOUT ===

Uncertainty Decomposition Summary:

Mean Epistemic Uncertainty: 0.00605

Mean Aleatoric Uncertainty: 0.15482

Mean Predictive Entropy: 0.63933

```
In [8]: # 1. Uncertainty as a function of missing leads
k_levels = [12, 10, 8, 6, 4, 3, 2, 1]
inc_masks_data = np.load(os.path.join(MET_DIR, "incomplete_lead_masks.npz"))

lead_deg_epistemic = []
lead_deg_aleatoric = []

model.train()
for k in k_levels:
    sc_name = 'full_12' if k == 12 else f'random_{k}_leads'
    mask_k = inc_masks_data[sc_name]

    k_probs = []
    k_logvars = []
    with torch.no_grad():
        for i in range(10): # 10 MC passes per missing lead condition
            l, lv = model(torch.tensor(X_test).to(device), torch.tensor(mask_k).to(
                k_probs.append(torch.sigmoid(l / optimal_T).cpu().numpy())
                k_logvars.append(lv.cpu().numpy())

    k_probs = np.array(k_probs)
    k_logvars = np.array(k_logvars)
    p_m = np.mean(k_probs, axis=0)

    epist = np.mean(np.var(k_probs, axis=0))
    aleat = np.mean(np.exp(k_logvars) * (p_m * (1.0 - p_m)))

    lead_deg_epistemic.append(epist)
    lead_deg_aleatoric.append(aleat)
model.eval()

# 2. Uncertainty vs Prediction Error (Selective Rejection)
test_preds = (p_mean >= 0.5).astype(int)
correct_mask = (test_preds == y_test) # (N, C)
correct_epistemic = u_epistemic[correct_mask]
error_epistemic = u_epistemic[~correct_mask]

# Plot Figure 14
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Subplot A: Uncertainty vs Missing Leads
axes[0].plot(k_levels, lead_deg_epistemic, 'o-', color='#1f77b4', lw=2.5, label='Ep')
axes[0].plot(k_levels, lead_deg_aleatoric, 's--', color='#ff7f0e', lw=2.5, label='A')
axes[0].set_xlabel('Number of Available Leads ($k$)')
axes[0].set_ylabel('Mean Uncertainty Value')
axes[0].set_title('A: Uncertainty vs. Number of Available Leads', fontweight='bold')
axes[0].invert_xaxis() # Show degradation from 12 down to 1
axes[0].legend(loc='upper right', frameon=True)
axes[0].grid(True, linestyle=':', alpha=0.6)

# Subplot B: Uncertainty Distribution for Correct vs Incorrect Predictions
```

```

sns.kdeplot(correct_epistemic, ax=axes[1], color='#2ca02c', fill=True, label=f'Correct')
sns.kdeplot(error_epistemic, ax=axes[1], color='#d62728', fill=True, label=f'Incorrect')
axes[1].set_xlabel('Epistemic Uncertainty')
axes[1].set_ylabel('Density')
axes[1].set_title('B: Epistemic Uncertainty vs. Diagnostic Error', fontweight='bold')
axes[1].legend(loc='upper right', frameon=True)
axes[1].grid(True, linestyle=':', alpha=0.6)

# Subplot C: Error Rejection Curve (Risk vs Coverage)
# Flatten across test instances
all_epistemic = u_epistemic.ravel()
all_errors = (~correct_mask).ravel().astype(float)
sort_idx = np.argsort(all_epistemic) # Low uncertainty to high uncertainty
sorted_errors = all_errors[sort_idx]

coverages = np.linspace(0.2, 1.0, 50)
error_rates = [np.mean(sorted_errors[:int(len(sorted_errors)*cov)]) for cov in coverages]

axes[2].plot(coverages * 100, np.array(error_rates) * 100, color='#9467bd', lw=2.5)
axes[2].set_xlabel('Data Coverage Retained (%)')
axes[2].set_ylabel('Error Rate (%)')
axes[2].set_title('C: Selective Classification (Risk-Coverage Tradeoff)', fontweight='bold')
axes[2].grid(True, linestyle=':', alpha=0.6)

plt.suptitle('LAC-SSM Uncertainty Decomposition & Clinical Rejection Analysis', fontweight='bold')
plt.tight_layout()
fig14_path = os.path.join(FIG_DIR, "fig14_uncertainty_quantification.png")
plt.savefig(fig14_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 14 saved to: {fig14_path}")

```

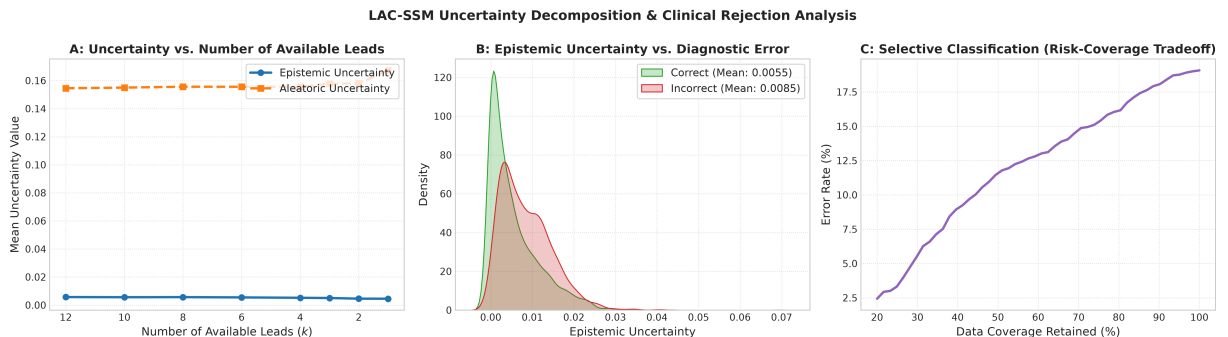


Figure 14 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig14_uncertainty_quantification.png